# BT CÁ NHÂN 3 • VT-117

Vũ Thành Công • 23001839 • Thành viên • SE2026-T29 • APP-03 Production Readiness Lab

Repository: https://github.com/CuongMotNang/SE2026-T29 (GitHub chuyển tới T-ng-D-c-Lieu-Dong/SE2026-T29). Baseline: 2a4bbc3b246e1f2c9b947a4d8de56489a4b37fc9. Nhánh: https://github.com/T-ng-D-c-Lieu-Dong/SE2026-T29/tree/bt3-23001839-vt117

Cam kết để sinh viên tự xác nhận trước khi nộp: “Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI.”

## 1. Specification và contract

Vấn đề: giá trị mặc định số appointment trên một trang được cố định trong API; người vận hành cần thay đổi giá trị nghiệp vụ này qua cấu hình tập trung mà không sửa mã ở nhiều nơi.

AC1: Không cấu hình thì mặc định 20. AC2: BOOKING_DEFAULT_PAGE_SIZE nhận số nguyên từ 1 đến 100; giá trị không rỗng ngoài miền bị từ chối khi nạp settings; biến rỗng được coi là không cấu hình. AC3: Khi không gửi limit, API dùng cấu hình và trả đúng limit, offset, total và items. AC4: limit hợp lệ do client gửi được ưu tiên; limit ngoài 1–100 trả 422. AC5: lỗi repository không được chuyển thành kết quả rỗng thành công.

Phạm vi: frontend/src/pages/BookingPage.tsx cố định PAGE_SIZE=20 và truyền limit tường minh, do đó UI hiện không dùng default cấu hình. Lát cắt này dành cho các client gọi API không truyền limit; không khẳng định giao diện đã thay đổi. Thay UI là yêu cầu mở rộng cần thống nhất với nhóm.

Invariant: phản hồi thành công phải phản ánh kết quả repository và phân trang thực; lỗi truy xuất không được che giấu bằng total=0. Ràng buộc: giữ contract query 1–100 và mặc định 20; cấu hình được cache trong process và yêu cầu khởi động lại; mục tiêu tải 120 tác vụ, p95 không quá 3,5 giây.

## 2. AI-assisted plan và review trước patch

Prompt tóm tắt: giải thích contract và invariant trước, đối chiếu requirements repository, lập plan và danh sách file; viết test fail trước, sửa nhỏ, kiểm tra mutation nuốt exception và tải 120 tác vụ. Giả định: cấu hình trang mặc định là một cấu hình nghiệp vụ phù hợp; không thay endpoint hay cấu trúc dữ liệu.

Hai điều chỉnh dựa trên repository: (1) không chọn SLOT_MINUTES: booking.py dùng nó để liệt kê slot và kiểm tra mốc 30 phút, thay riêng sẽ phá nhất quán; (2) không cấu hình mức trần 100: appointments.py và test_appointments_api.py đã công bố và kiểm thử cận này. Plan cuối sửa config.py, api/appointments.py, test contract và README; thêm harness và evidence.

Plan: đọc API, settings, service và tests; viết contract tests và lưu kết quả fail; thêm một field settings và dùng khi thiếu query limit; kiểm tra payload, mutation, tải, diff và regression. Files dự kiến: backend/app/config.py, backend/app/api/appointments.py, backend/tests/unit/test_page_size_contract.py và backend/README.md.

Giới hạn evidence: lịch sử còn prompt đề bài và phát biểu tóm tắt, nhưng không có transcript đầy đủ của prompt plan hoặc artifact plan lưu trước patch. Không thể chứng minh chi tiết thời điểm review của từng phương án bằng tài liệu có sẵn. Chọn page size là giả định cụ thể hóa change request chung, chưa được nhóm xác nhận.

Căn cứ đối chiếu: TASK-002 yêu cầu API đúng status và dữ liệu; test_appointments_api.py đã kiểm tra phân trang, limit 0/101. booking.py có quy tắc slot 30 phút. Hai điểm review nêu trên dựa vào repository chứ không xem AI là nguồn sự thật.

## 3. Chiến lược và test cases

Contract testing ở đây kiểm tra provider qua HTTP ASGI, gồm metadata và nội dung, cùng kiểm tra boundary settings; đây không phải consumer-driven contract có broker. Unit bao phủ validation và failure path; integration cần PostgreSQL thật; UI hiện luôn truyền limit=20, nên E2E UI không kiểm được nhánh default này; không giả vờ một smoke test UI xác nhận thay đổi. E2E hiện hữu có thể chạy để kiểm hồi quy, còn E2E cho cấu hình cần một consumer không truyền limit hoặc thay UI theo yêu cầu đã được duyệt. Không cần E2E cho từng giá trị biên vì lặp logic API và tăng chi phí.

| ID / ưu tiên | Risk / đầu vào | Kết quả / mức / trạng thái |
|---|---|---|
| T01 P1 | Không có biến môi trường | 20 / unit / pass |
| T02 P1 | Cấu hình 7 | Settings = 7 / unit / pass |
| T03 P1 | Biên 1 | Chấp nhận / unit / pass |
| T04 P1 | Biên 100 | Chấp nhận / unit / pass |
| T05 P1 | Cấu hình 0 | ValidationError / unit / pass |
| T06 P1 | Cấu hình 101 | ValidationError / unit / pass |
| T07 P1 | Cấu hình chữ invalid | ValidationError / unit / pass |
| T08 P1 | Không gửi limit, config 7, offset 1 | 200, limit 7, total 12, tên Lan / API giả / pass |
| T09 P1 | Gửi limit 3, config 7 | limit 3 và dữ liệu phân trang / API giả / pass |
| T10 P1 | limit 0 hoặc 101 | 422 / API giả / pass |
| T11 P1 | Repository raise RuntimeError | Exception lan truyền / unit / pass |
| T12 P1 | Hai booking, config 1 | Một mục, total 2 / PostgreSQL integration / pass |
| T13 P1 | Service ném lỗi khi GET | HTTP 500 / API giả / pass |
| T14 P2 | Biến môi trường rỗng | Default 20 / unit / pass |
| T15 P1 | Session lỗi trong repository thật | HTTP 500 / API giả session / pass |

Dữ liệu chọn theo lớp tương đương và biên: 0/1/100/101, chuỗi sai kiểu, giá trị khác mặc định 7 và limit tường minh 3. Fake trả total 12 và tên Lan để phát hiện test chỉ nhìn HTTP 200. T11 là failure path; T05–T07 và T10 là negative tests.

## 4. Test-first, diff và review patch

Baseline: 2a4bbc3. bt3_fail.log ghi 4 lỗi vì Settings chưa có default_page_size và không từ chối cấu hình sai, trước patch. Patch 6d9bf5f thêm Field(default=20, ge=1, le=100), chọn effective_limit khi query vắng, cập nhật README. Test xác nhận response thực tế; bộ unit sau mở rộng: 138 pass (bt3/unit_final.txt). PostgreSQL integration: 12 pass, bao gồm T12 (bt3/integration.txt).

Lệnh tái lập: python -m pytest backend/tests/unit/test_page_size_contract.py -q. Đối với PostgreSQL, đặt TEST_DATABASE_URL trỏ đúng booking_test rồi chạy python -m pytest backend/tests/integration/test_appointments_api.py -q. T12 tạo hai bản ghi qua POST, xác nhận đúng ID và thứ tự khi default=1, rồi limit=2 tường minh trả cả hai ID. Các lệnh đầy đủ ở bt3/README.md.

Diễn giải chuỗi bằng chứng: bt3_fail.log chạy bốn test đầu trước patch, có AttributeError cho field chưa tồn tại và ba trường hợp DID NOT RAISE. Commit 6d9bf5f có cả test và patch, nên fail trước patch được chứng minh bằng log, không phải một commit test-only. Giới hạn này làm audit độc lập yếu hơn việc có commit đỏ riêng.

Diff đi từ một literal 20 sang trường settings được validate, sau đó chọn limit do client truyền khi có; response giữ nguyên shape items/total/limit/offset. T12 dùng PostgreSQL test với hai bản ghi riêng; fixture TRUNCATE và DROP tables, vì vậy chỉ được chạy trên database dành cho test có tên kết thúc _test.

Review: query tường minh không bị thay thế; giá trị 0 bị FastAPI từ chối, không dùng phép “or” gây che lỗi; cấu hình cache nghĩa là đổi biến cần restart. OpenAPI không còn công bố default 20, một thay đổi mô tả contract cần nhóm review.

## 5. Mutation, coverage và flaky

Mutation tạm thời trong BookingService.list: bắt Exception rồi return [], 0. Chạy T11 cho kết quả DID NOT RAISE RuntimeError (bt3_mutation.log); đã khôi phục và không commit mutation. Test này chứng minh phát hiện một mutation cụ thể, không chứng minh mọi lỗi đều bị phát hiện.

Coverage chạy riêng 13 test contract với coverage.py --branch: config 90%, appointments 60%, booking service 33%, tổng 53% (bao gồm nhiều nhánh không thuộc lát cắt). Không coi tỷ lệ tổng là chất lượng: thiếu E2E và failure-path với DB thật là rủi ro quan trọng. Xem bt3/coverage.txt để biết dòng chưa chạy.

Phân tích branch: tổng báo cáo 150 statements, 62 chưa chạy; 20 branch opportunities, hai nhánh từng phần. Chỉ số theo ba module được import bởi 13 test chọn lọc, không phải coverage toàn repository. Các nhánh tạo/hủy appointment và nhiều nhánh validate telemetry nằm ngoài phạm vi lát cắt; branch coverage không đo được tính đúng của nội dung hay chất lượng assertion.

Risk coverage: AC1–AC4 được kiểm bằng biên, query và response; AC5 được kiểm ở service và ASGI khi fake ném lỗi. Chưa có test consumer OpenAPI, mất kết nối PostgreSQL thật hoặc thay đổi settings trong nhiều process. Mutation bị bắt cho thấy năng lực đối với lỗi cụ thể, không đại diện tất cả lỗi có thể có.

Flaky: (1) Thời gian: unit dùng ngày cố định 2030 và không phụ thuộc đồng hồ; test tích hợp dùng ngày tương lai động, còn rủi ro đổi ngày/múi giờ. (2) Shared state/thứ tự: get_settings dùng cache; test xóa cache trong finally và monkeypatch tự hoàn tác. Integration TRUNCATE dùng DB chuyên biệt; không chạy song song các test cùng DB. Thử chạy ba test theo thứ tự thay đổi: 3 pass, bt3/order_check.txt; đây là kiểm tra hạn chế, không chứng minh không có flaky. (3) Network và concurrency: ASGI in-process tránh mạng; benchmark vẫn chịu scheduler và không phản ánh contention DB.

## 6. Thử tải và giới hạn

Harness ASGI với repository giả chờ 10ms: 120/120 outcome đúng, p95 0,097715s. Phép đo bổ sung qua HTTP localhost, Uvicorn một worker và PostgreSQL 18 test container: 120 request đồng thời, 120/120 payload đúng, không lỗi, p95 nearest-rank 0,266092s, tổng thời gian 0,297382s. p95 là mẫu thứ ceil(0,95 × 120) = 114 sau khi sắp xếp độ trễ; các request đều được tính kể cả nếu lỗi.

Dữ liệu gồm hai dòng Load-A và Load-B, cấu hình mặc định 1. Mỗi response phải có limit=1, total=2, một item tên Load-A; harness chỉ seed và xóa hai ID trong DB test. So với ngưỡng 3,5s, p95 cục bộ thấp hơn 3,233908s. Lệnh: docker compose -f backend/compose.test.yaml up -d; đặt PYTHONPATH=backend; python bt3/load_postgres.py. Log: bt3/load_postgres_result.txt, commit 3618bd2.

Giới hạn: một burst, hai dòng, không warmup, không có mạng diện rộng, proxy hay tài nguyên production; client và server cùng máy. Phép đo không chứng minh SLO production hoặc throughput ổn định. Cần lặp nhiều lượt, dùng dữ liệu đại diện và theo dõi CPU, memory, pool cùng tỉ lệ lỗi.

## 7. Traceability và đối chiếu rubric

| AC / risk | Test → file | Commit / kết quả |
|---|---|---|
| AC1–2 | T01–T07, T14 → unit contract | 6d9bf5f, 29233a5, 5405940 / pass |
| AC3 | T08; T12 → unit; integration/test_appointments_api.py | 6d9bf5f; 5405940 / pass |
| AC4 | T09–T10, T12 → unit; integration | 29233a5; 5405940 / pass |
| AC5 / nuốt lỗi | T11, T13, T15 → unit contract | 6d9bf5f, 3618bd2, 5405940 / pass |
| 120, p95 | bt3/load_postgres.py; load_harness.py | 3618bd2 / cục bộ; giả lập |

Đường dẫn test contract đầy đủ: backend/tests/unit/test_page_size_contract.py. Bằng chứng và lệnh: bt3/README.md, bt3_fail.log, bt3_mutation.log, bt3/unit_final.txt, bt3/load_result.txt.

T13 (commit 3618bd2): service giả ném exception; ASGITransport không ném lại và test kiểm HTTP 500, tránh báo danh sách rỗng thành công. T11 kiểm lỗi service; T13 dùng fake service, T15 dùng repository thật với fake session ném lỗi, rồi kiểm HTTP 500. Chưa thử mất PostgreSQL thật hoặc UI.

Đối chiếu rubric: có 5 AC, constraints, invariant; 15 cases với negative và failure-path; fail → patch → pass; mutation bị bắt; coverage và flaky; 120 tác vụ và p95; traceability và AI log. Hạn chế: chưa lưu prompt/plan nguyên văn trước patch, chưa chạy E2E UI, tải chưa đại diện production. Không được suy diễn các mục này đã hoàn toàn đạt.

## 8. Reflection - bản nháp để sinh viên tự đối chiếu

Confidence tăng trước hết vì tôi có thể nối từng yêu cầu với một phép kiểm chứng cụ thể. Lần chạy đầu cho thấy settings chưa có trường cấu hình và không từ chối giá trị sai; sau patch, test tương ứng chuyển sang pass. Việc kiểm tra cả total, limit và tên khách hàng giúp phát hiện phản hồi có HTTP 200 nhưng sai nghiệp vụ. Mutation bắt mọi exception rồi trả danh sách rỗng cũng khiến test failure-path thất bại. Bằng chứng đó mạnh hơn việc chỉ đọc code hoặc nhìn tỷ lệ coverage, vì nó cho thấy một lỗi có chủ đích thực sự bị phát hiện. Tuy nhiên, một mutation bị bắt không có nghĩa suite phát hiện được mọi lỗi.

Tôi vẫn chưa thể khẳng định hệ thống production đạt mục tiêu hiệu năng. Con số p95 đã được đo qua HTTP localhost và PostgreSQL nhưng chỉ một burst với hai dòng; phép đo ASGI riêng dùng repository giả. Test PostgreSQL đã pass nhưng bộ test contract chưa bao phủ toàn bộ luồng UI; coverage tổng thấp phần nào vì các thao tác tạo và hủy không thuộc lát cắt đang đo. Ngoài ra cấu hình được cache, nên vận hành phải khởi động lại để thay đổi có hiệu lực; mô tả OpenAPI mặc định cũng thay đổi và cần review tương thích.

Test tiếp theo có giá trị cao nhất là chạy 120 request qua deployment đại diện, với PostgreSQL và dữ liệu đủ lớn, lặp lại nhiều lượt và kiểm tra cả nội dung lẫn lỗi, đồng thời đo p95 và tài nguyên. Trước khi nộp tôi cần tự đọc diff, chạy lại các lệnh, đối chiếu kết quả tích hợp thực tế và diễn đạt reflection theo hiểu biết của mình. Tôi không nên dùng kết quả giả lập như bằng chứng SLO production.

## Phụ lục A: AI Usage Log và evidence

Công cụ: Codex. Prompt nguồn: toàn bộ đề VT-117 trong cuộc trao đổi, sau đó yêu cầu “triển khai”, “tiếp tục triển khai” và kiểm tra rubric, đưa PDF lên branch. Prompt plan ở mục 2 là tóm tắt, không phải trích nguyên văn; không có bản ghi riêng chứng minh prompt plan đầy đủ được lưu trước code.

Output đã dùng: định hướng contract, patch settings và effective_limit, test, mutation tạm, harness và báo cáo nháp. Kiểm chứng bằng git diff, pytest, PostgreSQL, coverage.py và kết quả tải. Điều chỉnh: không chỉnh SLOT_MINUTES hay trần 100 khi chưa đàm phán contract; không kết luận SLO production từ ASGI. Evidence: TASK-002, appointments.py, booking.py, bt3_fail.log, bt3_mutation.log và log trong bt3/.

Hai điểm AI có vẻ hợp lý nhưng chưa đáng tin: chỉ assert HTTP 200 có thể lọt total/items sai, nên T08/T12/T13/T15 kiểm nội dung hoặc lỗi; dùng default động làm thay metadata OpenAPI dù runtime vẫn 20, cần review consumer. Cấu hình cache cũng yêu cầu restart. Các giả định cần nhóm và sinh viên tự xác nhận.

Lệnh evidence: python -m pytest backend/tests/unit -q (138 passed); python -m pytest backend/tests/integration/test_appointments_api.py -q với TEST_DATABASE_URL (12 passed); python -m coverage run --branch -m pytest backend/tests/unit/test_page_size_contract.py -q (13 passed); python bt3/load_postgres.py (120/120). Mã và log nằm trong nhánh; không dùng ảnh thay cho test.

Lưu ý: bản reflection và cam kết là bản nháp; sinh viên cần tự xác minh và chịu trách nhiệm trước khi nộp.